

Smart E-Commerce Project Summary

A full-stack e-commerce and blog platform built with React, FastAPI, and Django admin integration.

1. Project Overview

This project combines a customer-facing storefront with a secure FastAPI backend, user authentication, product catalog, shopping cart, checkout flow, order management, blog publishing, and notification services. It also includes a Django admin service for operational tasks and reporting.

Project Output

```
Pretty print ☒  
{  
  "message": "Smart E-Commerce API is running"  
}
```

2. Core Technologies

- Frontend: React + Vite + JavaScript
- Backend API: FastAPI + SQLAlchemy + Pydantic
- Database: MySQL for core commerce data and SQLite for blog data
- Authentication: JWT-based login with role support
- Payments: Stripe integration with mock/local checkout mode
- Admin: Django admin panel with analytics and reporting

3. Main Features

- User sign-up, login, JWT access control, and role-based authorization
- Product catalog browsing, filtering, and product details
- Shopping cart and summary management

- Checkout flow with payment intent creation and Stripe session handling
- Order lifecycle tracking and payment status updates
- In-app and WebSocket notifications with optional SMTP email alerts
- Public blog posts, article management, likes, comments, and category search
- Django admin panel for managing users, products, analytics, and exports

4. Repository Structure

- frontend/: React storefront and UI components
- fastapi_backend/: FastAPI app, database models, routes, services, and tests
- django_admin/: Django-based admin service and analytics dashboard
- postman/: API collection for manual testing
- README.md and module readmes: setup and deployment notes

5. Blog Management API

The blog module supports public post listing and filtering, slug-based retrieval, comment creation, post likes, draft management, and author/admin access control. Users can manage their own posts and admins can manage all posts, while public endpoints return published content only.

6. Setup and Run

- Create the MySQL database and configure environment variables in fastapi_backend/.env
- Install Python dependencies from fastapi_backend/requirements.txt
- Run uvicorn app.main:app --reload from fastapi_backend
- Install frontend dependencies from frontend and run npm run dev
- Optionally start the Django admin service on a separate port for staff operations

7. Expected Outcome

This project delivers a working e-commerce ecosystem with storefront UX, secure backend APIs, payment flows, blog publishing, admin oversight, and scalable module-based architecture suitable for extension into production.

Prepared from the repository documentation and API structure.